

1교시: Compose 기본과 검증 루프



Day 5: Compose Architecture Lab

로컬에서 실행하는 5가지 Docker Compose 아키텍처 패턴

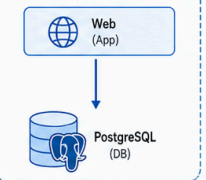
목표

- 실무에서 자주 쓰는 아키텍처 패턴을 Compose로 구성
- 서비스 간 통신, 볼륨, 네트워크를 이해
- 검증 루프로 반복하며 확실히 익히기

1 Web + DB

두 계층 웹 애플리케이션

compose 네트워크

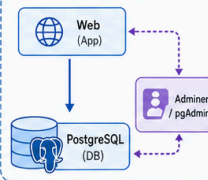


- 웹 애플리케이션과 PostgreSQL 구성
- 영속 볼륨으로 데이터 보존

2 DB 관리 UI

DB UI 도구 추가

compose 네트워크

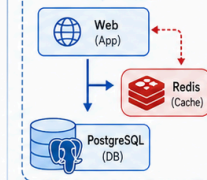


- Adminer 또는 pgAdmin으로 DB를 쉽게 관리

3 Redis Cache

캐시 계층 추가

compose 네트워크

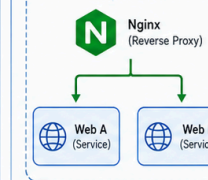


- Redis로 세션, 캐시, 카운터 등 처리
- 성능 향상 및 부하 분산

4 Reverse Proxy

Nginx로 두 웹 서비스 라우팅

compose 네트워크

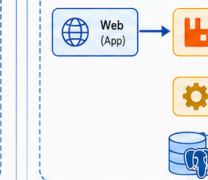


- Nginx가 라우팅/부하분산/SSL 종료
- 두 개의 웹 서비스를 외부에 노출

5 Queue + Worker

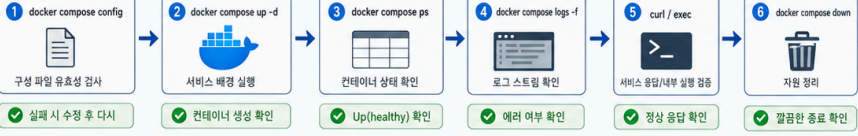
비동기 작업 처리 패턴

compose 네트워크



- 메시지 큐로 비동기 작업 처리
- 안정적인 백그라운드 작업 파이프라인

검증 루프



- 1. docker compose config: 구성 파일 유효성 검사 (실제 시 수정 후 다시)
- 2. docker compose up -d: 서비스 배포 실행 (컨테이너 생성 확인)
- 3. docker compose ps: 컨테이너 상태 확인 (Up(healthy) 확인)
- 4. docker compose logs -f: 로그 스트리밍 확인 (메러 여부 확인)
- 5. curl / exec: 서비스 응답/내부 실행 검증 (정상 응답 확인)
- 6. docker compose down: 자원 정리 (끝난 후 종료 확인)

실패 드릴

- 포트 충돌: 포트 변경 또는 중지
- 의존 서비스 미가동: depends_on, healthcheck 확인
- 현물/데이터 문제: 볼륨 마운트 경로, 권한 확인
- 네트워크 문제: 같은 네트워크 사용 확인
- 헬스체크 실패: 로그로 원인 파악 후 재시작

down vs down -v

docker compose down	docker compose down -v
컨테이너, 네트워크 제거	컨테이너, 네트워크 제거
볼륨(데이터)은 유지	역명 볼륨까지 삭제(데이터 삭제)

개발 초기에는 down -v 로 깔끔하게!
중요 데이터가 있다면 down 만 사용!

수업 목표

- Compose architecture를 실제 명령으로 실행한다.
- config/up/ps/logs/curl/exec/down 검증 루프를 적용한다.
- Week 3 MSA service boundary로 연결한다.

강의 전개

config/up/ps/logs/down 루프를 아키텍처 공통 기준으로 익힌다. 강사는 starter code와 compose.yaml 을 제공하고, 학생은 YAML을 읽은 뒤 실제로 실행해 서비스 관계를 확인한다. 유명 아키텍처를 말로만 설명하지 않고 container, network, volume, port, log evidence로 확인한다.

이 교시는 설명만 듣고 지나가지 않는다. 명령은 반드시 code block으로 실행하고, 바로 이어서 검증 명령을 실행한다. 정상 출력이 다를 수 있는 부분은 전체 문자열을 외우지 않고 성공 패턴을 기록한다. 실패도 수업 산출물이다. 실패한 명령, 에러 요약, 가설, 다시 확인한 명령을 함께 남긴다.

실습 명령

```
cd week2/day5/labs/compose-architectures/01-web-postgres
docker compose config
```

```
docker compose up -d
```

검증 명령

```
cd week2/day5/labs/compose-architectures/01-web-postgres
docker compose ps
docker compose logs --tail 80
```

```
cd week2/day5/labs/compose-architectures/01-web-postgres
# web가 있는 architecture는 curl로 확인
curl -I http://localhost:18085 || true
# DB가 있는 architecture는 exec 또는 run client로 확인
docker compose exec db psql -U postgres -d app -c 'SELECT 1;' || true
```

실패 드릴과 오해 교정

상황	해석
config 실패	indentation, env 누락, compose file path를 확인한다.
service unhealthy	logs와 dependency readiness를 확인한다.
down -v 남용	database volume 삭제 위험을 설명한다.

Cleanup

```
cd week2/day5/labs/compose-architectures/01-web-postgres
docker compose down
# 데이터를 초기화해도 되는 실습일 때만 실행
# docker compose down -v
```

Cleanup은 비용과 데이터 안전을 동시에 다룬다. container를 지우는 명령과 volume/network/image를 지우는 명령은 의미가 다르다. 특히 volume 삭제는 database data 삭제일 수 있으므로 실습 volume인지 확인한 뒤 실행한다.

Evidence

항목	제출 기준
Architecture	실행한 architecture 이름
Compose evidence	config/up/ps/logs/check 결과
Cleanup	down/down-v 판단
MSA 연결	service boundary와 장애 전파 해석

강의자 설명 포인트

Day 5는 Compose 문법 암기 시간이 아니라 architecture를 실행 가능한 파일로 제공하는 연습이다. Day 2의 volume/network, Day 3의 image, Day 4의 env/logs/cleanup이 Compose 한 파일 안에서 다시 만난다. 학생은 services, ports, environment, volumes, networks를 YAML 속성으로만 보지 말고 지난 실습의 docker run 옵션이 옮겨진 결과로 읽어야 한다.

유명한 아키텍처 패턴을 다룰 때도 그림만 보여주지 않는다. Web+DB, DB UI, cache, reverse proxy, queue+worker는 모두 실제 container로 띄워야 한다. docker compose config는 문법 검증이고, up은 시작이며, ps/logs/curl/exec가 정상 검증이다. down과 down -v는 cleanup과 data reset의 경계다.

운영 해석

Compose는 Kubernetes가 아니다. 하지만 Compose는 multi-service 사고를 배우기에 좋다. service name이 곧 내부 DNS가 되고, volume이 data lifecycle을 담당하며, ports가 host 공개 경계를 만든다. Week 3 MSA로 넘어갈 때 service boundary, dependency, failure propagation을 설명하는 첫 번째 실습 근거가 된다.

각 architecture는 반드시 실행 evidence를 남긴다. 학생이 두 개 이상의 architecture를 직접 실행하면 공통 패턴이 보이기 시작한다. 모든 architecture는 config, start, check, logs, cleanup이라는 같은 운영 루프를 가진다.

README 기록 예시

```
## Compose Architecture Evidence
- Architecture folder:
- Services:
- Config result:
- Up/ps result:
- HTTP or DB check:
- Logs summary:
```

- Volume/data cleanup decision:
- Failure drill:
- Week 3 MSA connection:

다음 연결

다음 architecture 또는 Week 3 MSA에서 같은 service/network/dependency 관점을 재사용한다.